UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 1.0 - Criar *Threads*

1. Conceito.

- As threads em Java são unidades de execução independentes dentro de um programa. Elas permitem que várias tarefas sejam realizadas concorrentemente (aparentando rodar ao mesmo tempo), aproveitando melhor o processador e tornando os programas mais responsivos.
    - Um programa Java normalmente começa com uma única thread, chamada thread principal (main).
    - Podemos criar novas threads para executar tarefas em paralelo com a thread principal.
    - Cada thread possui um ciclo de vida (criação, execução, espera, término).
    - O sistema operacional (junto com a JVM) realiza o agendamento (*scheduling*), decidindo qual thread roda em determinado instante.

- Ciclo de Vida de uma Thread.
    - Uma thread em Java passa pelos seguintes estados:
        - *New* (Nova): Criada, mas ainda não iniciada (new Thread()).
        - *Runnable* (Pronta): Pode ser executada, mas aguarda ser escalonada (start()).
        - *Running* (Executando): Está em execução no processador.
        - *Blocked / Waiting / Timed Waiting*: Aguardando recursos, outra thread, ou tempo.
        - *Terminated* (Finalizada): Terminou sua execução.
- Gerenciamento Moderno de Threads: Criar e gerenciar threads manualmente pode ser trabalhoso. Por isso, o Java oferece:
    - *Executors (ExecutorService)*: para gerenciar pools de threads.
    - *ForkJoinPool*: para tarefas que podem ser divididas em subtarefas.
    - *Parallel Streams*: processamento paralelo de coleções.
- Threads em Java permitem execução concorrente.
- São criadas por herança de **Thread** ou implementação de **Runnable**.
- O agendamento é feito pela JVM/SO.

2. Estender a classe Thread.

- Classe PrimeiraThread.java

```java
public class PrimeiraThread extends Thread{

	@Override
	public void run() {
		System.out.println("Método run executado por pela thread "
				+ Thread.currentThread().getName());
	}

	public static void main(String[] args) {

		Thread p = Thread.currentThread();
		System.out.println(p.getName());

		PrimeiraThread t0 = new PrimeiraThread();
		t0.run(); // executa na mesma thread
```

```
            //t0.start(); //chama o método run em nova thread

        }

}
```

- Classe SegundaThread.java

```java
public class SegundaThread extends Thread{

        @Override
        public void run() {
                System.out.println("Método run executado por pela thread "
                                + Thread.currentThread().getName());

                for (int i = 0; i < 5; i++) {
                        System.out.println(i);
                }

        }

        public static void main(String[] args) {

                SegundaThread t0 = new SegundaThread();
                SegundaThread t1 = new SegundaThread();
                t0.run();
                t1.run();

                //t0.start();
                //t1.start();

        }

}
```

3. Implementar a interface Runnable.

- Classe MinhaThread.java

```java
public class MinhaThread implements Runnable {

        /**
         * Toda classe que implementa Runnable, deve implementar o método run().
         * É o local onde a lógica da thread é inserida.
         */

        /** A interface Runnable oferece um contrato que torna
         * a classe que a implementa executável.
        * Há a opção de estender a classe Thread. No entanto, herdamos
        * um monte de métodos e só implementamos o run.
```

```
	* Runnable estabelece que a classe deve implementar somente o run
	* tornando-a uma opção mais compacta e compatível com polimorfismo */

	@Override
	public void run() {
		System.out.println("Oi, eu sou a "
				+ Thread.currentThread().getName());
	}

}
```

- Classe App.java

Java

```
public class App {

	public static void main(String[] args) {

		// Pega referência da thread principal

		Thread t = Thread.currentThread();

		/**
		 * Escreve o nome da thread principal na tela
		 * se vc digitar t. vão aparecer outros métodos
		 * que são  características dessa thread
		 */
		System.out.println(t.getName());

		/**
		 * Cria uma instância da classe MinhaThread que implementa Runnable.
		 */
		MinhaThread mt = new MinhaThread();
		Thread t0 = new Thread(mt);
		t0.run(); // executada pela main
		t0.start(); // executada em nova thread

		/**
		 * As threads são criadas e inicializadas usando a mesma
		 * referência mt.
		 */

		Thread t1 = new Thread(mt);
		t1.start();

		Thread t2 = new Thread(mt);
		t2.start();

	}

}
```

- Classe CriarThreadsApp.java
    - Formas de criar e lançar threads.

Java

```java
public class CriarThreadsApp {

	public static void main(String[] args) {

		Runnable r = new MinhaThread();
		Thread t0 = new Thread(r);
		t0.start();

		// Usando classes anônimas

		Runnable r1 = new Runnable() {

			@Override
			public void run() {
				System.out.println("Sou a thread "
						+ Thread.currentThread().getName());
			}
		};
		Thread t1 = new Thread(r1);
		t1.start();


		new Thread(new Runnable() {

			@Override
			public void run() {
				System.out.println("Sou a thread "
						+ Thread.currentThread().getName());

			}
		}).start();

		//Usando lambda

		Runnable r2 = () -> {

				System.out.println("Sou a thread "
						+ Thread.currentThread().getName());

		};
		Thread t2 = new Thread(r2);
		t2.start();

		Thread t3 = new Thread(

				() -> System.out.println("Sou a thread "
					+ Thread.currentThread().getName())
		);

		t3.start();
	}

}
```

4. Informações sobre as threads.

Java

```java
public class InfoThreads implements Runnable{

	@Override
	public void run() {

		Thread t = Thread.currentThread();

		System.out.println("Algumas informações da thread: "
				+ "\nNome: " + t.getName()
				+ "\nId: " + t.getId()
				+ "\nEstado: " + t.getState()
				+ "\nPrioridade: " + t.getPriority()
				+ "\nViva? " + t.isAlive()
				+ "\nInterrompida? " + t.isInterrupted()
				+ "\nDaemon? " + t.isDaemon());

	}

	public static void main(String[] args) {

		Runnable r0 = new InfoThreads();
		Thread t0 = new Thread(r0);
		System.out.println(t0.getState());
		t0.start();

	}

}
```